

1. Os seguintes tipos de dados em Haskell:

- (a) `Atom` que representa um átomo identificado por um nome.
- (b) `Tree a b` que representa uma árvore binária cujas folhas armazenam valores do tipo `b` e cujos nós internos armazenam valores do tipo `a`.
- (c) `Result a` um tipo que pode conter tanto um valor de tipo `a` quanto uma `String` com uma mensagem de erro.

2. Considere a solução para a questão b. Instancie o tipo de dados para a classe `Eq`, implementando apenas a função `==`.

3. Considere a seguinte definição de tipo de dados em Haskell.

```
data RoseTree a = Branch a [RoseTree a] deriving (Show,Eq)
```

- (a) Defina `mapRT :: (a -> b) -> RoseTree a -> RoseTree b`.
- (b) Defina `foldrRT :: (a -> [b] -> b) -> b -> RoseTree a -> b`.
- (c) Use `foldrRT` para expressar `mapRT`.
- (d) Use `foldrRT` para expressar `altura :: RoseTree a -> Int`, que computa a altura da árvore.

4. Considere o seguinte tipo de dados algébrico que descreve expressões da lógica proposicional.

```
data LE = And LE LE  
       | Or LE LE  
       | Not LE  
       | Const Bool  
       | Var String  
       deriving Show
```

- (a) Defina a função `eval :: [(String,Bool)] -> LE -> Maybe Bool` que, dado um contexto que associa nomes a valores booleanos, determina o valor lógico da expressão.
- (b) Defina a função `vars :: LE -> [String]` que retorna uma lista de todas as variáveis que ocorrem na fórmula.
- (c) Instancie o tipo de dados para a classe `Eq` de tal modo que duas expressões sejam iguais se, e somente se, suas representações forem idênticas.